

Assessment 6
L0 Final Assessment
Data Structures Coding

Name: Nisanth Saravanan
ID: 12576

Question:

Case Study: Hospital Management System

Overview: This C++ program simulates a basic hospital management system, focusing on array-based data storage and fundamental object-oriented principles. The program defines a Patient structure to organize patient information, including name, age, and diagnosis. A Hospital class encapsulates the patient management logic, utilizing a fixed-size array to store patient records. The Hospital class provides methods for

Hospital Management System:

1. Add Patient
2. Display Patients
3. Find Patient
4. Remove Patient
5. Exit

The program features a menu-driven interface, allowing users to interact with the system through console input. The program demonstrates array-based data management, basic object-oriented design, and fundamental C++ programming concepts within a simplified hospital management context.

CODE:

```
#include <bits/stdc++.h>
#include <string>
#include <array>

using namespace std;

//Patient Struct:
struct Patient
{
    int ID;
    string Name;
    int Age;
    string Diagnosis;

    Patient() {} //Empty constructor.

    Patient(int id, string name, int age, string diagnosis) //Parametrised constructor.
    {
        ID = id;
        Name = name;
        Age = age;
        Diagnosis = diagnosis;
    }
};

class Hospital //Class that encapsulates all the record management logics.
{
public: //Specifying access.
    array<Patient, 100> AddPatient(array<Patient, 100> patients, int &n)
    {
        if (n > 100)
        {
            cout << "\n===== " << endl;
            cout << "    RECORDS ARE FULL!" << endl;
            cout << "===== " << endl;
            return patients;
        }

        cout << "\nENTER PATIENT DETAILS:" << endl;
        cout << "ID: ";
        int id;
        cin >> id;
        cout << "NAME: ";
        string name;
        cin.ignore();
        getline(cin, name);
        cout << "AGE: ";
        int age;
        cin >> age;
        cout << "DIAGNOSIS: ";
        string diagnosis;
        cin.ignore();
        getline(cin, diagnosis);
    }
};
```

```

    Patient patient = Patient(id, name, age, diagnosis);
    n++;
    patients[n] = patient;
    cout << "PATIENT ADDED SUCCESSFULLY!" << endl;
    return patients;
}

void DisplayPatient(array<Patient, 100> patients, int &n)
{
    if (n == -1)
    {
        cout << "\n===== " << endl;
        cout << "    RECORDS ARE EMPTY!" << endl;
        cout << "===== " << endl;
        return;
    }

    cout << "\n===== " << endl;
    cout << "    PATIENTS:" << endl;
    cout << "===== " << endl;
    for (int i = 0; i <= n; i++)
    {
        cout << "\nID: " << patients[i].ID << endl;
        cout << "NAME: " << patients[i].Name << endl;
        cout << "AGE: " << patients[i].Age << endl;
        cout << "DIAGNOSIS: " << patients[i].Diagnosis << endl;
    }
}

array<Patient, 100> RemovePatient(array<Patient, 100> patients, int &n)
{
    if (n == -1)
    {
        cout << "\n===== " << endl;
        cout << "    RECORDS ARE EMPTY!" << endl;
        cout << "===== " << endl;
        return patients;
    }

    cout << "\nENTER PATIENT ID TO REMOVE:" << endl;
    int id;
    cin >> id;
    int index = -1;
    for (int i = 0; i <= n; i++)
    {
        if (patients[i].ID == id)
        {
            index = i;
            break;
        }
    }
    if (index != -1)
    {
        for (int i = index; i <= n; i++)
        {
            patients[i] = patients[i + 1];
        }
    }
}

```

```

        n--;
        cout << "PATIENT REMOVED SUCCESSFULLY!" << endl;
        return patients;
    }
    else
    {
        cout << "PATIENT NOT FOUND!" << endl;
        return patients;
    }
}

void FindPatient(array<Patient, 100> patients, int &n)
{
    if (n == -1)
    {
        cout << "\n===== " << endl;
        cout << "    RECORDS ARE EMPTY!" << endl;
        cout << "===== " << endl;
        return;
    }

    cout << "\nENTER PATIENT ID TO FIND:" << endl;
    int id;
    cin >> id;
    int index = -1;
    for (int i = 0; i <= n; i++)
    {
        if (patients[i].ID == id)
        {
            index = i;
            break;
        }
    }
    if (index != -1)
    {
        cout << "\n===== " << endl;
        cout << "    RESULTS:" << endl;
        cout << "===== " << endl;
        cout << "\nID: " << patients[index].ID << endl;
        cout << "NAME: " << patients[index].Name << endl;
        cout << "AGE: " << patients[index].Age << endl;
        cout << "DIAGNOSIS: " << patients[index].Diagnosis << endl;
    }
    else
    {
        cout << "PATIENT NOT FOUND!" << endl;
    }
}

};

int main() //Driver code.
{
    array<Patient, 100> patients;
    Hospital hospital;
    int n = -1;
    int choice;

```

checkpoint1: [//Label-Goto Mechanism.](#)

```
cout << "\n===== " << endl;
cout << "HOSPITAL MANAGEMENT SYSTEM" << endl;
cout << "===== " << endl;
cout << "1. ADD PATIENT." << endl;
cout << "2. DISPLAY PATIENTS." << endl;
cout << "3. FIND PATIENT." << endl;
cout << "4. REMOVE PATIENT." << endl;
cout << "0. EXIT." << endl;

cout << "\nENTER YOUR CHOICE:";
cin >> choice;

switch (choice)
{
case 0:
{
    cout << "\n===== " << endl;
    cout << "    LOGIN OUT!!" << endl;
    cout << "===== " << endl;
    return 0;
}

default:
{
    cout << "\n===== " << endl;
    cout << "INVALID CHOICE, TRY AGAIN!" << endl;
    cout << "===== " << endl;
    goto checkpoint1;
}

case 1:
{
    patients = hospital.AddPatient(patients, n);
    goto checkpoint1;
}

case 2:
{
    hospital.DisplayPatient(patients, n);
    goto checkpoint1;
}

case 3:
{
    hospital.FindPatient(patients, n);
    goto checkpoint1;
}

case 4:
{
    patients = hospital.RemovePatient(patients, n);
    goto checkpoint1;
}
}
```

OUTPUT:

Adding Patients:

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:1

ENTER PATIENT DETAILS:
ID: 1
NAME: Jack
AGE: 23
DIAGNOSIS: Cancer
PATIENT ADDED SUCCESSFULLY!
```

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:1

ENTER PATIENT DETAILS:
ID: 2
NAME: Danny
AGE: 10
DIAGNOSIS: Common Cold
PATIENT ADDED SUCCESSFULLY!
```

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:1

ENTER PATIENT DETAILS:
ID: 3
NAME: Charlie
AGE: 36
DIAGNOSIS: Diabeties
PATIENT ADDED SUCCESSFULLY!
```

Display Patients:

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:2

=====
PATIENTS:
=====

ID: 1
NAME: Jack
AGE: 23
DIAGNOSIS: Cancer

ID: 2
NAME: Danny
AGE: 10
DIAGNOSIS: Common Cold

ID: 3
NAME: Charlie
AGE: 36
DIAGNOSIS: Diabeties
```

Find Patients:

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:3

ENTER PATIENT ID TO FIND:
1

=====
RESULTS:
=====

ID: 1
NAME: Jack
AGE: 23
DIAGNOSIS: Cancer
```

Remove Patient with verification:

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:4

ENTER PATIENT ID TO REMOVE:
2
PATIENT REMOVED SUCCESSFULLY!
```



```
ENTER YOUR CHOICE:2
```

```
=====
PATIENTS:
=====
```

```
ID: 1
NAME: Jack
AGE: 23
DIAGNOSIS: Cancer
```

```
ID: 3
NAME: Charlie
AGE: 36
DIAGNOSIS: Diabeties
```

Exiting:

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:0

=====
LOGIN OUT!!
=====
```

Series of invalid inputs:

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:5

=====
INVALID CHOICE, TRY AGAIN!
=====
```

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:3

=====
RECORDS ARE EMPTY!
=====
```

Finding.

```
=====
HOSPITAL MANAGEMENT SYSTEM
=====
1. ADD PATIENT.
2. DISPLAY PATIENTS.
3. FIND PATIENT.
4. REMOVE PATIENT.
0. EXIT.

ENTER YOUR CHOICE:4

=====
RECORDS ARE EMPTY!
=====
```

Removal.